Chapter 9
Discretization

Abstract Discretization is an essential preprocessing technique used in many knowledge discovery and data mining tasks. Its main goal is to transform a set of continuous attributes into discrete ones, by associating categorical values to intervals and thus transforming quantitative data into qualitative data. An overview of discretization together with a complete outlook and taxonomy are supplied in Sects. 9.1 and 9.2. We conduct an experimental study in supervised classification involving the most representative discretizers, different types of classifiers, and a large number of data sets (Sect. 9.4).

9.1 Introduction

As it was mentioned in the introduction of this book, data usually comes in different formats, such as discrete, numerical, continuous, categorical, etc. Numerical data, provided by discrete or continuous values, assumes that the data is ordinal, there is an order among the values. However, in categorical data, no order can be assumed amongst them. The domain and type of data is crucial to the learning task to be performed next. For example, in a decision tree induction process a feature must be chosen from a subset based on some metric gain associated with its values. This process usually requires inherent finite values and also prefers to perform a branch of values that are not ordered. Obviously, the tree structure is a finite structure and there is a need to split the feature to produce the associated nodes in further divisions. If data is continuous, there is a need to discretize the features either before the decision tree induction or throughout the process of tree modelling.

Discretization, as one of the basic data reduction techniques, has received increasing research attention in recent years [75] and has become one of the preprocessing techniques most broadly used in DM. The discretization process transforms quantitative data into qualitative data, that is, numerical attributes into discrete or nominal attributes with a finite number of intervals, obtaining a non-overlapping partition of a continuous domain. An association between each interval with a numerical discrete value is then established. In practice, discretization can be viewed as a data reduction method since it maps data from a huge spectrum of numeric values to a greatly

S. García et al., *Data Preprocessing in Data Mining*,
Intelligent Systems Reference Library 72, DOI 10.1007/978-3-319-10247-4_9

reduced subset of discrete values. Once the discretization is performed, the data can be treated as nominal data during any induction or deduction DM process. Many existing DM algorithms are designed only to learn in categorical data, using nominal attributes, while real-world applications usually involve continuous features. Numerical features have to be discretized before using such algorithms.

In supervised learning, and specifically classification, the topic of this survey, we can define the discretization as follows. Assuming a data set consisting of N examples and C target classes, a discretization algorithm would discretize the continuous attribute A in this data set into m discrete intervals $D = \{[d_0, d_1], (d_1, d_2], \ldots, (d_{m-1}, d_m]\}$, where d_0 is the minimal value, d_m is the maximal value and $d_i < d_{i+i}$, for $i = 0, 1, \ldots, m-1$. Such a discrete result D is called a discretization scheme on attribute A and $P = \{d_1, d_2, \ldots, d_{m-1}\}$ is the set of cut points of attribute A.

The necessity of using discretization on data can be caused by several factors. Many DM algorithms are primarily oriented to handle nominal attributes [36, 75, 123], or may even only deal with discrete attributes. For instance, three of the ten methods considered as the top ten in DM [120] require an embedded or an external discretization of data: C4.5 [92], Apriori [1] and Naïve Bayes [44, 122]. Even with algorithms that are able to deal with continuous data, learning is less efficient and effective [29, 94]. Other advantages derived from discretization are the reduction and the simplification of data, making the learning faster and yielding more accurate, with compact and shorter results; and any noise possibly present in the data is reduced. For both researchers and practitioners, discrete attributes are easier to understand, use, and explain [75]. Nevertheless, any discretization process generally leads to a loss of information, making the minimization of such information loss the main goal of a discretizer.

Obtaining the optimal discretization is NP-complete [25]. A vast number of discretization techniques can be found in the literature. It is obvious that when dealing with a concrete problem or data set, the choice of a discretizer will condition the success of the posterior learning task in accuracy, simplicity of the model, etc. Different heuristic approaches have been proposed for discretization, for example, approaches based on information entropy [36, 41], statistical χ^2 test [68, 76], likelihood [16, 119], rough sets [86, 124], etc. Other criteria have been used in order to provide a classification of discretizers, such as univariate/multivariate, supervised/unsupervised, top-down/bottom-up, global/local, static/dynamic and more. All these criteria are the basis of the taxonomies already proposed and they will be deeply elaborated upon in this chapter. The identification of the best discretizer for each situation is a very difficult task to carry out, but performing exhaustive experiments considering a representative set of learners and discretizers could help to make the best choice.

Some reviews of discretization techniques can be found in the literature [9, 36, 75, 123]. However, the characteristics of the methods are not studied completely, many discretizers, even classic ones, are not mentioned, and the notation used for categorization is not unified. In spite of the wealth of literature, and apart from the absence of a complete categorization of discretizers using a unified notation, it can be observed that, there are few attempts to empirically compare them. In this way, the algorithms proposed are usually compared with a subset of the complete family of discretizers

and, in most of the studies, no rigorous empirical analysis has been carried out. In [51], it was noticed that the most compared techniques are EqualWidth, EqualFrequency, MDLP [41], ID3 [92], ChiMerge [68], 1R [59], D2 [19] and Chi2 [76].

These reasons motivate the global purpose of this chapter. We can summarize it into three main objectives:

- To provide an updated and complete taxonomy based on the main properties observed in the discretization methods. The taxonomy will allow us to characterize their advantages and drawbacks in order to choose a discretizer from a theoretical point of view.
- To make an empirical study analyzing the most representative and newest discretizers in terms of the number of intervals obtained and inconsistency level of the data.
- Finally, to relate the best discretizers to a set of representative DM models using two metrics to measure the predictive classification success.

9.2 Perspectives and Background

Discretization is a wide field and there have been many advances and ideas over the years. This section is devoted to provide a proper background on the topic, together with a set of related areas and future perspectives on discretization.

9.2.1 Discretization Process

Before starting, we must first introduce some terms used by different sources for the sake of unification.

9.2.1.1 Feature

Also called *attribute* or *variable* refers to an aspect of the data and it is usually associated to the columns in a data table. M stands for the number of features in the data.

9.2.1.2 Instance

Also called *tuple*, *example*, *record* or *data point* refers to a collection of feature values for all features. A set of instances constitute a data set and they are usually associated to row in a data table. According to the introduction, we will set N as the number of instances in the data.

9.2.1.3 Cut Point

Refers to a real value that divides the range into two adjacent intervals, being the first interval less than or equal to the cut point and the second interval greater than the cut point.

9.2.1.4 Arity

In discretization context, it refers to the number of partitions or intervals. For example, if the arity of a feature after being discretized is m, then there will be $m-1$ cut points.

We can associate a typical discretization as an univariate discretization. Although this property will be reviewed in Sect. 9.3.1, it is necessary to introduce it here for the basic understanding of the basic discretization process. Univariate discretization operates with one continuous feature at a time while multivariate discretization considers multiple features simultaneously.

A typical discretization process generally consists of four steps (seen in Fig. 9.1): (1) *sorting* the continuous values of the feature to be discretized, (2) *evaluating* a cut

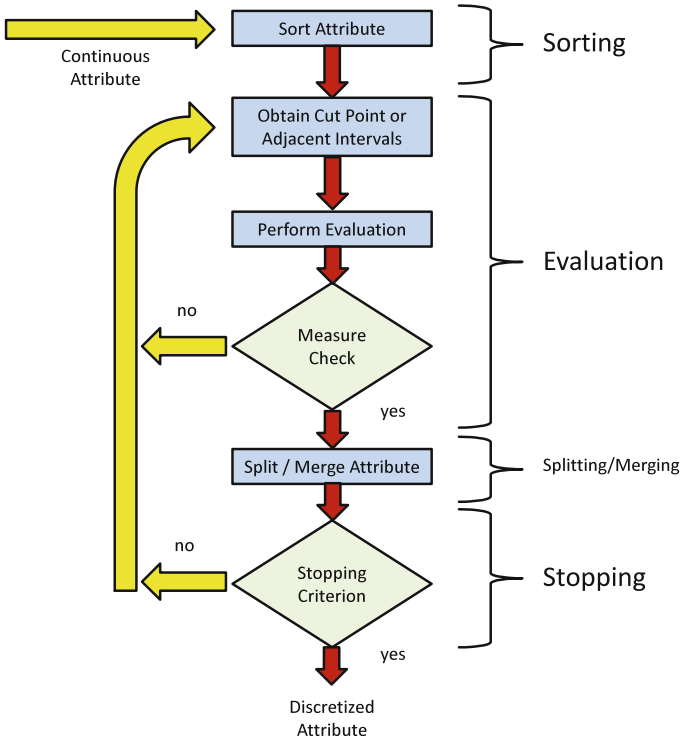

Fig. 9.1 Discretization process

point for splitting or adjacent intervals for merging, (3) *splitting or merging* intervals of continuous values according to some defined criterion, and (4) *stopping* at some point. Next, we explain these four steps in detail.

9.2.1.5 Sorting

The continuous values for a feature are sorted in either descending or ascending order. It is crucial to use an efficient sorting algorithm with a time complexity of $O(NlogN)$, for instance the well-known *Quick Sort* algorithm. Sorting must be done only once and for all the start of discretization. It is a mandatory treatment and can be applied when the complete instance space is used for discretization. However, if the discretization is within the process of other algorithms (such as decision trees induction), it is a local treatment and only a region of the whole instance space is considered for discretization.

9.2.1.6 Selection of a Cut Point

After sorting, the best cut point or the best pair of adjacent intervals should be found in the attribute range in order to split or merge in a following required step. An evaluation measure or function is used to determine the correlation, gain, improvement in performance and any other benefit according to the class label. There are numerous evaluation measures and they will be discussed in Sect. 9.3.1, the entropy and the statistical dependency being the most well known.

9.2.1.7 Splitting/Merging

Depending on operation method of the discretizers, intervals either can be split or merged. For splitting all the possible cut points from the whole universe within an attribute must be evaluated. The universe is formed from all the different real values presented in an attribute. Then, the best one is found and a split of the continuous range into two partitions is performed. Discretization continues with each part until a stopping criterion is satisfied. Similarly for merging, instead of finding the best cut point, the discretizer aims to find the best adjacent intervals to merge in each iteration. Discretization continues with the reduced number of intervals until the stopping criterion is satisfied.

9.2.1.8 Stopping Criteria

It specifies when to stop the discretization process. It should assume a trade-off between lower arity getting a better understanding or simplicity with high accuracy

or consistency. For example, a threshold for m can be an upper bound for the arity of the resulting discretization. A stopping criterion can be very simple such as fixing the number of final intervals at the beginning of the process or more complex like estimating a function.

9.2.2 Related and Advanced Work

Research in improving and analyzing discretization is common and in high demand currently. Discretization is a promising technique to obtain the hoped results, depending on the DM task, which justifies its relationship to other methods and problems. This section provides a brief summary of topics closely related to discretization from a theoretical and practical point of view and describes other works and future trends which have been studied in the last few years.

- *Discretization Specific Analysis:* Susmaga proposed an analysis method for discretizers based on binarization of continuous attributes and rough sets measures [104]. He emphasized that his analysis method is useful for detecting redundancy in discretization and the set of cut points which can be removed without decreasing the performance. Also, it can be applied to improve existing discretization approaches.
- *Optimal Multisplitting:* Elomaa and Rousu characterized some fundamental properties for using some classic evaluation functions in supervised univariate discretization. They analyzed entropy, information gain, gain ratio, training set error, Gini index and normalized distance measure, concluding that they are suitable for use in the optimal multisplitting of an attribute [38]. They also developed an optimal algorithm for performing this multisplitting process and devised two techniques [39, 40] to speed it up.
- *Discretization of Continuous Labels:* Two possible approaches have been used in the conversion of a continuous supervised learning (regression problem) into a nominal supervised learning (classification problem). The first one is simply to use regression tree algorithms, such as CART [17]. The second consists of applying discretization to the output attribute, either statically [46] or in a dynamic fashion [61].
- *Fuzzy Discretization:* Extensive research has been carried out around the definition of linguistic terms that divide the domain attribute into fuzzy regions [62]. Fuzzy discretization is characterized by membership value, group or interval number and affinity corresponding to an attribute value, unlike crisp discretization which only considers the interval number [95].
- *Cost-Sensitive Discretization:* The objective of cost-based discretization is to take into account the cost of making errors instead of just minimizing the total sum of errors [63]. It is related to problems of imbalanced or cost-sensitive classification [57, 103].

- *Semi-Supervised Discretization:* A first attempt to discretize data in semi-supervised classification problems has been devised in [14], showing that it is asymptotically equivalent to the supervised approach.

The research mentioned in this section is out of the scope of this book. We point out that the main objective of this chapter is to give a wide overview of the discretization methods found in the literature and to conduct an experimental comparison of the most relevant discretizers without considering external and advanced factors such as those mentioned above or derived problems from classic supervised classification.

9.3 Properties and Taxonomy

This section presents a taxonomy of discretization methods and the criteria used for building it. First, in Sect. 9.3.1, the main characteristics which will define the categories of the taxonomy will be outlined. Then, in Sect. 9.3.2, we enumerate the discretization methods proposed in the literature and we will consider by using both their complete name and abbreviated name together with the associated reference. Finally, we present the taxonomy.

9.3.1 Common Properties

This section provides a framework for the discussion of the discretizers presented in the next subsection. The issues discussed include several properties involved in the structure of the taxonomy, since they are exclusive to the operation of the discretizer. Other, less critical issues such as parametric properties or stopping conditions will be presented although they are not involved in the taxonomy. Finally, some criteria will also be pointed out in order to compare discretization methods.

9.3.1.1 Main Characteristics of a Discretizer

In [36, 51, 75, 123], various axes have been described in order to make a categorization of discretization methods. We review and explain them in this section, emphasizing the main aspects and relations found among them and unifying the notation. The taxonomy presented will be based on these characteristics:

- *Static versus Dynamic:* This characteristic refers to the moment and independence which the discretizer operates in relation to the learner. A dynamic discretizer acts when the learner is building the model, thus they can only access partial information (local property, see later) embedded in the learner itself, yielding compact and accurate results in conjuntion with the associated learner. Otherwise, a static discretizer proceeds prior to the learning task and it is independent from

the learning algorithm [75]. Almost all known discretizers are static, due to the fact that most of the dynamic discretizers are really subparts or stages of DM algorithms when dealing with numerical data [13]. Some examples of well-known dynamic techniques are ID3 discretizer [92] and ITFP [6].

- *Univariate versus Multivariate:* Multivariate techniques, also known as 2D discretization [81], simultaneously consider all attributes to define the initial set of cut points or to decide the best cut point altogether. They can also discretize one attribute at a time when studying the interactions with other attributes, exploiting high order relationships. By contrast, univariate discretizers only work with a single attribute at a time, once an order among attributes has been established, and the resulting discretization scheme in each attribute remains unchanged in later stages. Interest has recently arisen in developing multivariate discretizers since they are very influential in deductive learning [10, 49] and in complex classification problems where high interactions among multiple attributes exist, which univariate discretizers might obviate [42, 121].
- *Supervised versus Unsupervised:* Unsupervised discretizers do not consider the class label whereas supervised ones do. The manner in which the latter consider the class attribute depends on the interaction between input attributes and class labels, and the heuristic measures used to determine the best cut points (entropy, interdependence, etc.). Most discretizers proposed in the literature are supervised and theoretically using class information, should automatically determine the best number of intervals for each attribute. If a discretizer is unsupervised, it does not mean that it cannot be applied over supervised tasks. However, a supervised discretizer can only be applied over supervised DM problems. Representative unsupervised discretizers are EqualWidth and EqualFrequency [73], PKID and FFD [122] and MVD [10].
- *Splitting versus Merging:* This refers to the procedure used to create or define new intervals. Splitting methods establish a cut point among all the possible boundary points and divide the domain into two intervals. By contrast, merging methods start with a pre-defined partition and remove a candidate cut point to mix both adjacent intervals. These properties are highly related to *Top-Down* and *Bottom-up* respectively (explained in the next section). The idea behind them is very similar, except that top-down or bottom-up discretizers assume that the process is incremental (described later), according to a hierarchical discretization construction. In fact, there can be discretizers whose operation is based on splitting or merging more than one interval at a time [72, 96]. Also, some discretizers can be considered *hybrid* due to the fact that they can alternate splits with merges in running time [24, 43].
- *Global versus Local:* To make a decision, a discretizer can either require all available data in the attribute or use only partial information. A discretizer is said to be local when it only makes the partition decision based on local information. Examples of widely used local techniques are MDLP [41] and ID3 [92]. Few discretizers are local, except some based on top-down partition and all the dynamic techniques. In a top-down process, some algorithms follow the divide-and-conquer scheme and when a split is found, the data is recursively divided, restricting access

to partial data. Regarding dynamic discretizers, they find the cut points in internal operations of a DM algorithm, so they never gain access to the full data set.

- *Direct versus Incremental:* Direct discretizers divide the range into k intervals simultaneously, requiring an additional criterion to determine the value of k. They do not only include one-step discretization methods, but also discretizers which perform several stages in their operation, selecting more than a single cut point at every step. By contrast, incremental methods begin with a simple discretization and pass through an improvement process, requiring an additional criterion to know when to stop it. At each step, they find the best candidate boundary to be used as a cut point and afterwards the rest of the decisions are made accordingly. Incremental discretizers are also known as hierarchical discretizers [9]. Both types of discretizers are widespread in the literature, although there is usually a more defined relationship between incremental and supervised ones.
- *Evaluation Measure:* This is the metric used by the discretizer to compare two candidate schemes and decide which is more suitable to be used. We consider five main families of evaluation measures:
  - *Information:* This family includes *entropy* as the most used evaluation measure in discretization (MDLP [41], ID3 [92], FUSINTER [126]) and other derived information theory measures such as the *Gini index* [66].
  - *Statistical:* Statistical evaluation involves the measurement of dependency/correlation among attributes (Zeta [58], ChiMerge [68], Chi2 [76]), probability and bayesian properties [119] (MODL [16]), interdependency [70], contingency coefficient [106], etc.
  - *Rough Sets:* This group is composed of methods that evaluate the discretization schemes by using rough set measures and properties [86], such as lower and upper approximations, class separability, etc.
  - *Wrapper:* This collection comprises methods that rely on the error provided by a classifier that is run for each evaluation. The classifier can be a very simple one, such as a majority class voting classifier (Valley [108]) or general classifiers such as Naïve Bayes (NBIterative [87]).
  - *Binning:* This category refers to the absence of an evaluation measure. It is the simplest method to discretize an attribute by creating a specified number of bins. Each bin is defined as a priori and allocates a specified number of values per attribute. Widely used binning methods are EqualWidth and EqualFrequency.

9.3.1.2 Other Properties

We can discuss other properties related to discretization which also influence the operation and results obtained by a discretizer, but to a lower degree than the characteristics explained above. Furthermore, some of them present a large variety of categorizations and may harm the interpretability of the taxonomy.

- *Parametric versus Non-Parametric:* This property refers to the automatic determination of the number of intervals for each attribute by the discretizer. A nonpara-

metric discretizer computes the appropriate number of intervals for each attribute considering a trade-off between the loss of information or consistency and obtaining the lowest number of them. A parametric discretizer requires a maximum number of intervals desired to be fixed by the user. Examples of non-parametric discretizers are MDLP [41] and CAIM [70]. Examples of parametric ones are ChiMerge [68] and CADD [24].

- *Top-Down versus Bottom Up:* This property is only observed in incremental discretizers. Top-Down methods begin with an empty discretization. Its improvement process is simply to add a new cutpoint to the discretization. On the other hand, Bottom-Up methods begin with a discretization that contains all the possible cutpoints. Its improvement process consists of iteratively merging two intervals, removing a cut point. A classic Top-Down method is MDLP [41] and a well-known Bottom-Up method is ChiMerge [68].
- *Stopping Condition:* This is related to the mechanism used to stop the discretization process and must be specified in nonparametric approaches. Well-known stopping criteria are the Minimum Description Length measure [41], confidence thresholds [68], or inconsistency ratios [26].
- *Disjoint versus Non-Disjoint:* Disjoint methods discretize the value range of the attribute into disassociated intervals, without overlapping, whereas non-disjoint methods dicsretize the value range into intervals that can overlap. The methods reviewed in this chapter are disjoint, while fuzzy discretization is usually non-disjoint [62].
- *Ordinal versus Nominal:* Ordinal discretization transforms quantitative data into ordinal qualitative data whereas nominal discretization transforms it into nominal qualitative data, discarding the information about order. Ordinal discretizers are less common, and not usually considered classic discretizers [80].

9.3.1.3 Criteria to Compare Discretization Methods

When comparing discretization methods, there are a number of criteria that can be used to evaluate the relative strengths and weaknesses of each algorithm. These include the number of intervals, inconsistency, predictive classification rate and time requirements

- *Number of Intervals:* A desirable feature for practical discretization is that discretized attributes have as few values as possible, since a large number of intervals may make the learning slow and ineffective [19].
- *Inconsistency:* A supervision-based measure used to compute the number of unavoidable errors produced in the data set. An unavoidable error is one associated to two examples with the same values for input attributes and different class labels. In general, data sets with continuous attributes are consistent, but when a discretization scheme is applied over the data, an inconsistent data set may be obtained. The desired inconsistency level that a discretizer should obtain is 0.0.

- *Predictive Classification Rate:* A successful algorithm will often be able to discretize the training set without significantly reducing the prediction capability of learners in test data which are prepared to treat numerical data.
- *Time requirements:* A static discretization process is carried out just once on a training set, so it does not seem to be a very important evaluation method. However, if the discretization phase takes too long it can become impractical for real applications. In dynamic discretization, the operation is repeated as many times as the learner requires, so it should be performed efficiently.

9.3.2 Methods and Taxonomy

At the time of writting, more than 80 discretization methods have been proposed in the literature. This section is devoted to enumerating and designating them according to a standard followed in this chapter. We have used 30 discretizers in the experimental study, those that we have identified as the most relevant ones. For more details on their descriptions, the reader can visit the URL associated to the KEEL project.[1] Additionaly, implementations of these algorithms in Java can be found in KEEL software [3, 4].

Table 9.1 presents an enumeration of discretizers reviewed in this chapter. The complete name, abbreviation and reference are provided for each one. This chapter does not collect the descriptions of the discretizers. Instead, we recommend that readers consult the original references to understand the complete operation of the discretizers of interest. Discretizers used in the experimental study are depicted in bold. The ID3 discretizer used in the study is a static version of the well-known discretizer embedded in C4.5.

The properties studied above can be used to categorize the discretizers proposed in the literature. The seven characteristics studied allows us to present the taxonomy of discretization methods in an established order. All techniques enumerated in Table 9.1 are collected in the taxonomy drawn in Fig. 9.2. It illustrates the categorization following a hierarchy based on this order: static/dynamic, univariate/multivariate, supervised/unsupervised, splitting/merging/hybrid, global/local, direct/incremental and evaluation measure. The rationale behind the choice of this order is to achieve a clear representation of the taxonomy.

The proposed taxonomy assists us in the organization of many discretization methods so that we can classify them into categories and analyze their behavior. Also, we can highlight other aspects in which the taxonomy can be useful. For example, it provides a snapshot of existing methods and relations or similarities among them. It also depicts the size of the families, the work done in each one and what is currently missing. Finally, it provides a general overview of the state-of-the-art methods in discretization for researchers/practitioners who are beginning in this field or need to discretize data in real applications.

[1] http://www.keel.es.

Table 9.1 Discretizers

Complete name	Abbr. name	Reference	Complete name	Abbr. name	Reference
Equal Width Discretizer	**EqualWidth**	[115]	*No name specified*	Butterworth04	[18]
Equal Frequency Discretizer	**EqualFrequency**	[115]	*No name specified*	Zhang04	[124]
No name specified	Chou91	[27]	**Khiops**	**Khiops**	[15]
Adaptive Quantizer	AQ	[21]	**Class-Attribute Interdependence Maximization**	**CAIM**	[70]
Discretizer 2	D2	[19]	**Extended Chi2**	**Extended Chi2**	[101]
ChiMerge	**ChiMerge**	[68]	**Heterogeneity Discretizer**	**Heter-Disc**	[78]
One-Rule Discretizer	**1R**	[59]	**Unsupervised Correlation Preserving Discretizer**	**UCPD**	[81]
Iterative Dichotomizer 3 Discretizer	**ID3**	[92]	*No name specified*	Multi-MDL	[42]
Minimum Description Length Principle	**MDLP**	[41]	Difference Similitude Set Theory Discretizer	DSST	[116]
Valley	Valley	[108, 109]	Multivariate Interdependent Discretizer	MIDCA	[22]
Class-Attribute Dependent Discretizer	**CADD**	[24]	**MODL**	**MODL**	[16]
ReliefF Discretizer	ReliefF	[69]	Information Theoretic Fuzzy Partitioning	ITFP	[6]
Class-driven Statistical Discretizer	StatDisc	[94]	*No name specified*	Wu06	[118]
No name specified	NBIterative	[87]	Fast Independent Component Analysis	FastICA	[67]
Boolean Reasoning Discretizer	BRDisc	[86]	Linear Program Relaxation	LP-Relaxation	[37]
Minimum Description Length Discretizer	MDL-Disc	[89]	**Hellinger-Based Discretizer**	**HellingerBD**	[72]
Bayesian Discretizer	**Bayesian**	[119]	**Distribution Index-Based Discretizer**	**DIBD**	[117]
No name specified	Friedman96	[45]	Wrapper Estimation of Distribution Algorithm	WEDA	[43]
Cluster Analysis Discretizer	**ClusterAnalysis**	[26]	Clustering + Rought Sets Discretizer	Cluster-RS-Disc	[100]
Zeta	**Zeta**	[58]	**Interval Distance Discretizer**	**IDD**	[96]
Distance-based Discretizer	**Distance**	[20]	**Class-Attribute Contingency Coefficient**	**CACC**	[106]
Finite Mixture Model Discretizer	FMM	[102]	Rectified Chi2	Rectified Chi2	[91]

(continued)

Table 9.1 (continued)

Chi2	**Chi2**	[76]	**Ameva**	**Ameva**	[53]
No name specified	FischerExt	[127]	Unification	Unification	[66]
Contextual Merit Numerical Feature Discretizer	CM-NFD	[60]	Multiple Scanning Discretizer	MultipleScan	[54]
Concurrent Merger	ConMerge	[110]	Optimal Flexible Frequency Discretizer	OFFD	[111]
Knowledge EXplorer Discretizer	KEX-Disc	[11]	**Proportional Discretizer**	**PKID**	[122]
LVQ-based Discretization	LVQ-Disc	[88]	**Fixed Frequency Discretizer**	**FFD**	[122]
No name specified	Multi-Bayesian	[83]	Discretization Class intervals Reduce	DCR	[90]
No name specified	A*	[47]	MVD-CG	MVD-CG	[112]
FUSINTER	**FUSINTER**	[126]	Approximate Equal Frequency Discretizer	AEFD	[65]
Cluster-based Discretizer	Cluster-Disc	[82]	*No name specified*	Jiang09	[65]
Entropy-based Discretization According to Distribution of Boundary points	EDA-DB	[5]	Random Forest Discretizer	RFDisc	[12]
No name specified	Clarke00	[30]	Supervised Multivariate Discretizer	SMD	[64]
Relative Unsupervised Discretizer	RUDE	[79]	Clustering Based Discretization	CBD	[49]
Multivariate Discretization	**MVD**	[10]	Improved MDLP	Improved MDLP	[74]
Modified Learning from Examples Module	MODLEM	[55]	Imfor-Disc	Imfor-Disc	[125]
Modified Chi2	**Modified Chi2**	[105]	Clustering ME-MDL	Cluster ME-MDL	[56]
HyperCluster Finder	HCF	[84]	Effective Bottom-up Discretizer	EBDA	[98]
Entropy-based Discretization with Inconsistency Checking	EDIC	[73]	Contextual Discretizer	Contextual-Disc	[85]
Unparametrized Supervised Discretizer	**USD**	[52]	**Hypercube Division Discretizer**	**HDD**	[121]
Rough Set Discretizer	RS-Disc	[34]	Range Coefficient of Dispersion and Skewness	DRDS	[7]
Rough Set Genetic Algorithm Discretizer	RS-GA-Disc	[23]	Entropy-based Discretization using Scope of Classes	EDISC	[99]
Genetic Algorithm Discretizer	GA-Disc	[33]	Universal Discretizer	UniDis	[97]
Self Organizing Map Discretizer	SOM-Disc	[107]	Maximum AUC-based discretizer	MAD	[71]
Optimal Class-Dependent Discretizer	OCDD	[77]			

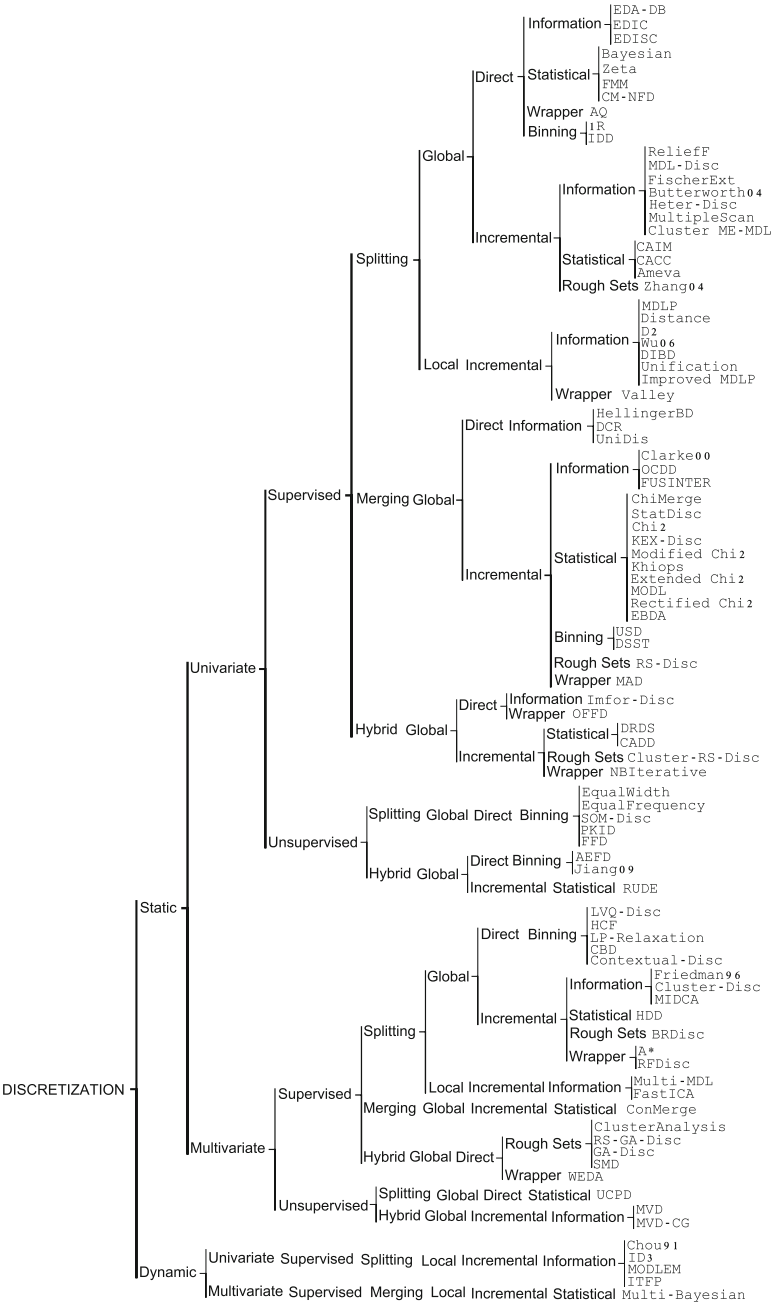

Fig. 9.2 Discretization taxonomy

9.3.3 Description of the Most Representative Discretization Methods

This section is devoted to provide an in-depth description of the most representative methods according to the previous taxonomy, the degree of usage in recent years in the specialized literature and the results are reported in the following section of this chapter, related to the experimental comparative analysis. We will discuss 10 discretizers in more detail: Equal Width/Frequency, MDLP, ChiMerge, Distance, Chi2, PKID, FFD, FUSINTER, CAIM and Modified Chi2. They will be distributed according to the splitting or merging criterion, which can be explained separately to the rest of mechanisms of each discretizer because it is usually a shared process.

9.3.3.1 Splitting Methods

We start with a generalized pseudocode for splitting discretization methods.

Algorithm 1 Splitting Algorithm

```
Require: S = Sorted values of attribute A
  procedure SPLITTING(S)
    if StoppingCriterion() == true then
      Return
    end if
    T = GetBestSplitPoint(S)
    S1 = GetLeftPart(S,T)
    S2 = GetRightPart(S,T)
    Splitting(S1)
    Splitting(S2)
  end procedure
```

The splitting algorithm above consists of all four steps in the discretization scheme, *sort* the feature values, (2) *search* for an appropriate cut point, (3) *split* the range of continuous values according to the cut point, and (4) *stop* when a stopping criterion satisfies, otherwise go to (2). In the following, we include the description of the most representative discretizers based on splitting criterion.

Equal Width or Frequency [75]
They belong to the simplest family of methods that discretize an attribute by creating a specified number of bins, which are created by equal width or equal frequency. This family is known as *Binning* methods. The arity m must be specified at the beginning of both discretizers and determine the number of bins. Each bin is associated with a different discrete value. In *equal width*, the continuous range of a feature is divided into intervals that have an equal width and each interval represents a bin. The arity can be calculated by the relationship between the chosen width for each interval and the total length of the attribute range. In *equal frequency*, an equal number of

continuous values are placed in each bin. Thus, the width of each interval is computed by dividing the length of the attribute range by the desired arity.

There is no need for a stopping criterion as the number of bins is computed directly at the beginning of the process. In practice, both methods are equivalent as they only depend on the number of bins desired, regardless of the calculation method.

MDLP [41]
This discretizer uses the entropy measure to evaluate candidate cut points. Entropy is one of the most commonly used discretization measures in the literature. The entropy of a sample variable X is

$$H(X) = -\sum_{x} p_x \log p_x$$

where x represents a value of X and p_x its estimated probability of occurring. It corresponds to the average amount of information per event where information of an event is defines as:

$$I(x) = -\log p_x$$

Information is high for lower probable events and low otherwise. This discretizer uses the *Information Gain* of a cut point, which is defined as

$$G(A, T; S) = H(S) - H(A, T; S) = H(S) - \frac{|S_1|}{N} H(S_1) - \frac{|S_2|}{N} H(S_2)$$

where A is the attribute in question, T is a candidate cut point and S is the set of N examples. So, S_i is a partitioned subset of examples produced by T.

The MDLP discretizer applies the *Minimum Description Length Principle* to decide the acceptation or rejection for each cut point and to govern the stopping criterion. It is defined in information theory to be the minimum number of bits required to uniquely specify an object out of the universe of all objects. It computes a final cost of coding and takes part in the decision making of the discretizer.

In summary, the MDLP criterion is that the partition induced by a cut point T for a set S of N examples is accepted iff

$$G(A, T; S) > \frac{\log_2(N-1)}{N} + \frac{\delta(A, T; S)}{N}$$

where $\delta(A, T; S) = \log_2(3^c - 2) - [c \cdot H(S) - c_1 \cdot H(S_1) - c_2 \cdot H(S_2)]$. We recall that c stands for the number of classes of the data set in supervised learning.

The stopping criterion is given in the MDLP itself, due to the fact that $\delta(A, T; S)$ acts as a threshold to stop accepting new partitions.

Distance [20]
This method introduces a distance measure called *Mantaras Distance* to evaluate the cut points. Let us consider two partitions S_a and S_b on a range of continuous values,

each containing c_a and C_b number of classes. The Mantaras distance between two partitions due to a single cut point is given below.

$$Dist(S_a, S_b) = \frac{I(S_a|S_b) + I(S_b|S_a)}{I(S_a \cap S_b)}$$

Since,

$$I(S_b|S_a) = I(S_b \cap S_a) - I(S_a)$$
$$Dist(S_a, S_b) = 2 - \frac{I(S_a) + I(S_b)}{I(S_a \cap S_b)}$$

where,

$$I(S_a) = -\sum_{i=1}^{c_a} S_i \log_2 S_i$$
$$I(S_b) = -\sum_{j=1}^{c_b} S_j \log_2 S_j$$
$$I(S_a \cap S_b) = -\sum_{i=1}^{c_a}\sum_{j=1}^{c_b} S_{ij} \log_2 S_{ij}$$
$$S_i = \frac{|C_i|}{N}$$
$$|C_i| = \text{ total count of class } i$$
$$N = \text{ total number of instances}$$
$$S_{ij} = S_i \times S_j$$

It chooses the cut point that minimizes the distance. As a stopping criterion, it uses the minimum description length discussed previously to determine whether more cut points should be added.

PKID [122]
In order to maintain a low bias and a low variance in a learning scheme, it is recommendable to increase both the interval frequency and the number of intervals as the amount of training data increases too. A good way to achieve this is to set interval frequency and interval number equally proportional to the amount of training data. This the main purpose of *proportional discretization* (PKID).

When discretizing a continuous attribute for which there are N instances, supposing that the desired interval frequency is s and the desired interval number is t, PKID calculates s and t by the following expressions:

$$s \times t = n$$
$$s = t$$

Thus, this discretizer is an equal frequency (or equal width) discretizer where both the interval frequency and number of intervals have the same quantity and they only depend on the number of instances in training data.

FFD [122]
FFD stands for *fixed frequency discretization* and was proposed for managing bias and variance especially in naive-bayes based classifiers. To discretize a continuous attribute, FFD sets a *sufficient interval frequency*, f. Then it discretizes the ascendingly sorted values into intervals of frequency f. Thus each interval has approximately the same number f of training instances with adjacent values. Incorporating f, FFD aims to ensure that the interval frequency is sufficient so that there are enough training instances in each interval to reliably estimate the Naïve Bayes probabilities.

There may be confusion when distinguishing equal frequency discretization from FFD. The former one fixes the interval number, thus it arbitrarily chooses the interval number and then discretizes a continuous attribute into intervals such that each interval has the same number of training instances. On the other hand, the later method, FFD, fixes the interval frequency by the value f. It then sets cut points so that each interval contains f training instances, controlling the discretization variance.

CAIM [70]
CAIM stands for Class-Attribute Interdependency Maximization criterion, which measures the dependency between the class variable C and the discretized variable D for attibute A. The method requires the computation of the quanta matrix [24], which, in summary, collects a snapshot of the number of real values of A within each interval and for each class of the corresponding example. The criterion is calculated as:

$$CAIM(C, D, A) = \frac{\sum_{r=1}^{n} \frac{\max_r^2}{M_{+r}}}{m}$$

where m is the number of intervals, r iterates through all intervals, i.e. $r = 1, 2, \ldots, m$, $\max_r$ is the maximum value among all q_{ir} values (maximum value within the rth column of the quanta matrix), M_{+r} is the total number of continuous values of attribute A that are within the interval $(d_{r-1}, d_r]$.

According to the authors, CAIM has the following properties:

- The larger the value of CAIM, the higher the interdependence between the class labels and the discrete intervals.
- It generates discretization schemes where each interval has all of its values grouped within a single class label.
- It has taken into account the negative impact that values belonging to classes, other than the class with the maximum number of values within an interval, have on the discretization scheme.

- The algorithm favors discretization schemes with a smaller number of intervals.

The stopping criterion is satisfied when the CAIM value in a new iteration is equal to or less than the CAIM value of the past iteration or if the number of intervals is less than the number of classes of the problem.

9.3.3.2 Merging Methods

We start with a generalized pseudocode for merging discretization methods.

Algorithm 2 Merging Algorithm

```
Require: S = Sorted values of attribute A
  procedure MERGING(S)
    if StoppingCriterion() == true then
      Return
    end if
    T = GetBestAdjacentIntervals(S)
    S = MergeAdjacentIntervals(S,T)
    Merging(S)
  end procedure
```

The merging algorithm above consists of four steps in the discretization process: (1) *sort* the feature values, (2) *search* for the best two neighboring intervals, (3) *merge* the pair into one interval, and (4) *stop* when a stopping criterion satisfies, otherwise go to (2). In the following, we include the description of the most representative discretizers based on merging criteria.

ChiMerge [68]
First of all, we introduce the χ^2 measure as other important evaluation metric in discretization. χ^2 is a statistical measure that conducts a significance test on the relationship between the values of an attribute and the class. The rationale behind this is that in accurate discretization, the relative class frequencies should be fairly consistent within an interval but two adjacent intervals should not have similar relative class frequency. The χ^2 statistic determines the similarity of adjacent intervals based on some significance level. Actually, it tests the hypothesis that two adjacent intervals of an attribute are independent of the class. If they are independent, they should be merged; otherwise they should remain separate. χ^2 is computed as:

$$\chi^2 = \sum_{i=1}^{2} \sum_{j=1}^{c} \frac{(N_{ij} - E_{ij})^2}{E_{ij}}$$

where:

$$
\begin{aligned}
c &= \text{number of classes} \\
N_{ij} &= \text{number of distinct values in the } i\text{th interval, } j\text{th class} \\
R_i &= \text{number of examples in } i\text{th interval } = \sum_{j=1}^{c} N_{ij} \\
C_j &= \text{number of examples in } j\text{th class } = \sum_{i=1}^{m} N_{ij} \\
N &= \text{total number of examples } = \sum_{j=1}^{c} C_j \\
E_{ij} &= \text{expected frequency of } N_{ij} = (R_i \times C_j)/N
\end{aligned}
$$

ChiMerge is a supervised, bottom-up discretizer. At the beginning, each distinct value of the attribute is considered to be one interval. χ^2 tests are performed for every pair of adjacent intervals. Those adjacent intervals with the least χ^2 value are merged until the chosen stopping criterion is satisfied. The significance level for χ^2 is an input parameter that determines the threshold for the stopping criterion. Another parameter used is the called *max-interval* which can be included to avoid the excessive number of intervals from being created. The recommended value for the significance level should be included between the range from 0.90 to 0.99. The *max-interval* parameter should be set to 10 or 15.

Chi2 [76]
It can be explained as an automated version of ChiMerge. Here, the statistical significance level keeps changing to merge more and more adjacent intervals as long as an inconsistency criterion is satisfied. We understand inconsistency to be two instances that match but belong to different classes. It is even possible to completely remove an attribute because the inconsistency property does not appear during the process of discretizing an attribute, acting as a feature selector. Like ChiMerge, χ^2 statistic is used to discretize the continuous attributes until some inconsistencies are found in the data.

The stopping criterion is achieved when there are inconsistencies in the data considering a limit of zero or δ inconsistency level as default.

Modified Chi2 [105]
In the original Chi2 algorithm, the stopping criterion was defined as the point at which the inconsistency rate exceeded a predefined rate δ. The δ value could be given after some tests on the training data for different data sets. The modification proposed was to use the level of consistency checking coined from Rough Sets Theory. Thus, this level of consistency replaces the basic inconsistency checking, ensuring that the fidelity of the training data could be maintained to be the same after discretization and making the process completely automatic.

Another fixed problem is related to the merging criterion used in Chi2, which was not very accurate, leading to overmerging. The original Chi2 algorithm computes the χ^2 value using an initially predefined degree of freedom (number of classes minus one). From the view of statistics, this was inaccurate because the degree of freedom may change according to the two adjacent intervals to be merged. This fact may change the order of merging and benefit the inconsistency after the discretization.

FUSINTER [126]
This method uses the same strategy as the ChiMerge method, but rather than trying to merge adjacent intervals locally, FUSINTER tries to find the partition which optimizes the measure. Next, we provide a short description of the FUSINTER algorithm.

- Obtain the boundary cut points after an increasing sorting of the values and the formation of intervals with run of examples of the same class. Superposition of several classes into an unique cut point is also allowed.
- Construct a matrix with a similar structure like a quanta matrix.
- Find two adjacent intervals whose merging would improve the value of the criterion and check if they can be merged using a differential criterion.
- Repeat until no improvement is possible or only an interval is achieved.

Two criteria can be used for deciding the merging of intervals. The first is the Shannon's entropy and the second is the quadratic entropy.

9.4 Experimental Comparative Analysis

This section presents the experimental framework and the results collected and discussions on them. Sect. 9.4.1 will describe the complete experimental set up. Then, we offer the study and analysis of the results obtained over the data sets used in Sect. 9.4.2.

9.4.1 Experimental Set up

The goal of this section is to show all the properties and issues related to the experimental study. We specify the data sets, validation procedure, classifiers used, parameters of the classifiers and discretizers, and performance metrics. Data sets and statistical tests used to contrast were described in the Chap. 2 of this book. Here, we will only specify the name of the data sets used. The performance of discretization algorithms is analyzed by using 40 data sets taken from the UCI ML Database Repository [8] and KEEL data set repository [3]. They are enumerated in Table 9.2.

In this study, six classifiers have been used in order to find differences in performance among the discretizers. The classifiers are:

Table 9.2 Enumeration of data sets used in the experimental study

Data set	Data set
Abalone	Appendicitis
Australian	Autos
Balance	Banana
Bands	Bupa
Cleveland	Contraceptive
Crx	Dermatology
Ecoli	Flare
Glass	Haberman
Hayes	Heart
Hepatitis	Iris
Mammographic	Movement
Newthyroid	Pageblocks
Penbased	Phoneme
Pima	Saheart
Satimage	Segment
Sonar	Spambase
Specfheart	Tae
Titanic	Vehicle
Vowel	Wine
Wisconsin	Yeast

- *C4.5* [92]: A well-known decision tree, considered one of the top 10 DM algorithms [120].
- *DataSqueezer* [28]: This learner belongs to the family of inductive rule extraction. In spite of its relative simplicity, DataSqueezer is a very effective learner. The rules generated by the algorithm are compact and comprehensible, but accuracy is to some extent degraded in order to achieve this goal.
- *KNN*: One of the simplest and most effective methods based on similarities among a set of objects. It is also considered one of the top 10 DM algorithms [120] and it can handle nominal attributes using proper distance functions such as HVDM [114]. It belongs to the lazy learning family [2, 48].
- *Naïve Bayes*: This is another of the top 10 DM algorithms [120]. Its aim is to construct a rule which will allow us to assign future objects to a class, assuming independence of attributes when probabilities are established.
- *PUBLIC* [93]: It is an advanced decision tree that integrates the pruning phase with the building stage of the tree in order to avoid the expansion of branches that would be pruned afterwards.
- *Ripper* [32]: This is a widely used rule induction method based on a *separate and conquer* strategy. It incorporates diverse mechanisms to avoid overfitting and to handle numeric and nominal attributes simultaneously. The models obtained are in the form of decision lists.

Table 9.3 Parameters of the discretizers and classifiers

Method	Parameters
C4.5	Pruned tree, confidence = 0.25, 2 examples per leaf
DataSqueezer	Pruning and generalization threshold = 0.05
KNN	$K = 3$, HVDM distance
PUBLIC	25 nodes between prune
Ripper	$k = 2$, grow set = 0.66
1R	6 examples of the same class per interval
CADD	Confidence threshold = 0.01
Chi2	Inconsistency threshold = 0.02
ChiMerge	Confidence threshold = 0.05
FDD	Frequency size = 30
FUSINTER	$\alpha = 0.975$, $\lambda = 1$
HDD	Coefficient = 0.8
IDD	Neighborhood = 3, windows size = 3, nominal distance
MODL	Optimized process type
UCPD	Intervals = [3, 6], KNN map type, neighborhood = 6,
	Minimum support = 25, merged threshold = 0.5,
	Scaling factor = 0.5, use discrete

The data sets considered are partitioned using the 10-FCV procedure. The parameters of the discretizers and classifiers are those recommended by their respective authors. They are specified in Table 9.3 for those methods which require them. We assume that the choice of the values of parameters is optimally chosen by their own authors. Nevertheless, in discretizers that require the input of the number of intervals as a parameter, we use a rule of thumb which is dependent on the number of instances in the data set. It consists in dividing the number of instances by 100 and taking the maximum value between this result and the number of classes. All discretizers and classifiers are run one time in each partition because they are non-stochastic.

Two performance measures are widely used because of their simplicity and successful application when multi-class classification problems are dealt with. We refer to accuracy and Cohen's kappa [31] measures, which will be adopted to measure the efficacy discretizers in terms of the generalization classification rate. The explanation of Cohen's kappa was given in Chap. 2.

The empirical study involves 30 discretization methods from those listed in Table 9.1. We want to outline that the implementations are only based on the descriptions and specifications given by the respective authors in their papers.

Table 9.4 Average results collected from intrinsic properties of the discretizers: number of intervals obtained and inconsistency rates in training and test data

Number int.		Incons. train		Incons. tst	
Heter-Disc	8.3125	ID3	0.0504	ID3	0.0349
MVD	18.4575	PKID	0.0581	PKID	0.0358
Distance	23.2125	Modified Chi2	0.0693	FFD	0.0377
UCPD	35.0225	FFD	0.0693	HDD	0.0405
MDLP	36.6600	HDD	0.0755	Modified Chi2	0.0409
Chi2	46.6350	USD	0.0874	USD	0.0512
FUSINTER	59.9850	ClusterAnalysis	0.0958	Khiops	0.0599
DIBD	64.4025	Khiops	0.1157	ClusterAnalysis	0.0623
CADD	67.7100	EqualWidth	0.1222	EqualWidth	0.0627
ChiMerge	69.5625	EqualFrequency	0.1355	EqualFrequency	0.0652
CAIM	72.5125	Chi2	0.1360	Chi2	0.0653
Zeta	75.9325	Bayesian	0.1642	FUSINTER	0.0854
Ameva	78.8425	MODL	0.1716	MODL	0.0970
Khiops	130.3000	FUSINTER	0.1735	HellingerBD	0.1054
1R	162.1925	HellingerBD	0.1975	Bayesian	0.1139
EqualWidth	171.7200	IDD	0.2061	UCPD	0.1383
Extended Chi2	205.2650	ChiMerge	0.2504	ChiMerge	0.1432
HellingerBD	244.6925	UCPD	0.2605	IDD	0.1570
EqualFrequency	267.7250	CAIM	0.2810	CAIM	0.1589
PKID	295.9550	Extended Chi2	0.3048	Extended Chi2	0.1762
MODL	335.8700	Ameva	0.3050	Ameva	0.1932
FFD	342.6050	1R	0.3112	CACC	0.2047
IDD	349.1250	CACC	0.3118	1R	0.2441
Modified Chi2	353.6000	MDLP	0.3783	Zeta	0.2454
CACC	505.5775	Zeta	0.3913	MDLP	0.2501
ClusterAnalysis	1116.1800	MVD	0.4237	DIBD	0.2757
USD	1276.1775	Distance	0.4274	Distance	0.2987
Bayesian	1336.0175	DIBD	0.4367	MVD	0.3171
ID3	1858.3000	CADD	0.6532	CADD	0.5688
HDD	2202.5275	Heter-Disc	0.6749	Heter-Disc	0.5708

9.4.2 Analysis and Empirical Results

Table 9.4 presents the average results corresponding to the number of intervals and inconsistency rate in training and test data by all the discretizers over the 40 data sets. Similarly, Tables 9.5 and 9.6 collect the average results associated to accuracy and kappa measures for each classifier considered. For each metric, the discretizers are ordered from the best to the worst. In Tables 9.5 and 9.6, we highlight those

Table 9.5 Average results of accuracy considering the six classifiers

C4.5		*DataSqueezer*		*KNN*		*Naïve Bayes*		*PUBLIC*		*Ripper*	
FUSINTER	0.7588	Distance	0.5666	PKID	0.7699	PKID	0.7587	FUSINTER	0.7448	Modified Chi2	0.7241
ChiMerge	0.7494	CAIM	0.5547	FFD	0.7594	Modified Chi2	0.7578	CAIM	0.7420	Chi2	0.7196
Zeta	0.7488	Ameva	0.5518	Modified Chi2	0.7573	FUSINTER	0.7576	ChiMerge	0.7390	PKID	0.7097
CAIM	0.7484	MDLP	0.5475	EqualFrequency	0.7557	ChiMerge	0.7543	MDLP	0.7334	MODL	0.7089
UCPD	0.7447	Zeta	0.5475	Khiops	0.7512	FFD	0.7535	Distance	0.7305	FUSINTER	0.7078
Distance	0.7446	ChiMerge	0.5472	EqualWidth	0.7472	CAIM	0.7535	Zeta	0.7301	Khiops	0.6999
MDLP	0.7444	CACC	0.5430	FUSINTER	0.7440	EqualWidth	0.7517	Chi2	0.7278	FFD	0.6970
Chi2	0.7442	Heter-Disc	0.5374	ChiMerge	0.7389	Zeta	0.7507	UCPD	0.7254	EqualWidth	0.6899
Modified Chi2	0.7396	DIBD	0.5322	CAIM	0.7381	EqualFrequency	0.7491	Modified Chi2	0.7250	EqualFrequency	0.6890
Ameva	0.7351	UCPD	0.5172	MODL	0.7372	MODL	0.7479	Khiops	0.7200	CAIM	0.6870
Khiops	0.7312	MVD	0.5147	HellingerBD	0.7327	Chi2	0.7476	Ameva	0.7168	HellingerBD	0.6816
MODL	0.7310	FUSINTER	0.5126	Chi2	0.7267	Khiops	0.7455	HellingerBD	0.7119	USD	0.6807
EqualFrequency	0.7304	Bayesian	0.4915	USD	0.7228	USD	0.7428	EqualFrequency	0.7110	ChiMerge	0.6804
EqualWidth	0.7252	Extended Chi2	0.4913	Ameva	0.7220	ID3	0.7381	MODL	0.7103	ID3	0.6787

(continued)

Table 9.5 (continued)

HellingerBD	0.7240	Chi2	0.4874	ID3	0.7172	Ameva	0.7375	CACC	0.7069	Zeta	0.6786
CACC	0.7203	HellingerBD	0.4868	ClusterAnalysis	0.7132	Distance	0.7372	DIBD	0.7002	HDD	0.6700
Extended Chi2	0.7172	MODL	0.4812	Zeta	0.7126	MDLP	0.7369	EqualWidth	0.6998	Ameva	0.6665
DIBD	0.7141	CADD	0.4780	HDD	0.7104	ClusterAnalysis	0.7363	Extended Chi2	0.6974	UCPD	0.6651
FFD	0.7091	EqualFrequency	0.4711	UCPD	0.7090	HellingerBD	0.7363	HDD	0.6789	CACC	0.6562
PKID	0.7079	1R	0.4702	MDLP	0.7002	HDD	0.7360	FFD	0.6770	Extended Chi2	0.6545
HDD	0.6941	EqualWidth	0.4680	Distance	0.6888	UCPD	0.7227	PKID	0.6758	Bayesian	0.6521
USD	0.6835	IDD	0.4679	IDD	0.6860	Extended Chi2	0.7180	USD	0.6698	ClusterAnalysis	0.6464
ClusterAnalysis	0.6813	USD	0.4651	Bayesian	0.6844	CACC	0.7176	Bayesian	0.6551	MDLP	0.6439
ID3	0.6720	Khiops	0.4567	CACC	0.6813	Bayesian	0.7167	ClusterAnalysis	0.6477	Distance	0.6402
1R	0.6695	Modified Chi2	0.4526	DIBD	0.6731	DIBD	0.7036	ID3	0.6406	IDD	0.6219
Bayesian	0.6675	HDD	0.4308	1R	0.6721	IDD	0.6966	MVD	0.6401	Heter-Disc	0.6084
IDD	0.6606	ClusterAnalysis	0.4282	Extended Chi2	0.6695	1R	0.6774	IDD	0.6352	1R	0.6058
MVD	0.6499	PKID	0.3942	MVD	0.6062	MVD	0.6501	1R	0.6332	DIBD	0.5953
Heter-Disc	0.6443	ID3	0.3896	Heter-Disc	0.5524	Heter-Disc	0.6307	Heter-Disc	0.6317	MVD	0.5921
CADD	0.5689	FFD	0.3848	CADD	0.5064	CADD	0.5669	CADD	0.5584	CADD	0.4130

Table 9.6 Average results of kappa considering the six classifiers

C4.5		*DataSqueezer*		*KNN*		*Naïve Bayes*		*PUBLIC*		*Ripper*	
FUSINTER	0.5550	CACC	0.2719	PKID	0.5784	PKID	0.5762	CAIM	0.5279	Modified Chi2	0.5180
ChiMerge	0.5433	Ameva	0.2712	FFD	0.5617	Modified Chi2	0.5742	FUSINTER	0.5204	Chi2	0.5163
CAIM	0.5427	CAIM	0.2618	Modified Chi2	0.5492	FUSINTER	0.5737	ChiMerge	0.5158	MODL	0.5123
Zeta	0.5379	ChiMerge	0.2501	Khiops	0.5457	FFD	0.5710	MDLP	0.5118	FUSINTER	0.5073
MDLP	0.5305	FUSINTER	0.2421	EqualFrequency	0.5438	ChiMerge	0.5650	Distance	0.5074	Khiops	0.4939
UCPD	0.5299	UCPD	0.2324	EqualWidth	0.5338	Chi2	0.5620	Zeta	0.5010	PKID	0.4915
Ameva	0.5297	Zeta	0.2189	CAIM	0.5260	CAIM	0.5616	Ameva	0.4986	EqualFrequency	0.4892
Chi2	0.5290	USD	0.2174	FUSINTER	0.5242	EqualWidth	0.5593	Chi2	0.4899	ChiMerge	0.4878
Distance	0.5288	Distance	0.2099	ChiMerge	0.5232	Khiops	0.5570	UCPD	0.4888	EqualWidth	0.4875
Modified Chi2	0.5163	Khiops	0.2038	MODL	0.5205	EqualFrequency	0.5564	Khiops	0.4846	CAIM	0.4870
MODL	0.5131	HDD	0.2030	HellingerBD	0.5111	MODL	0.5564	CACC	0.4746	Ameva	0.4810
EqualFrequency	0.5108	EqualFrequency	0.2016	Chi2	0.5100	USD	0.5458	HellingerBD	0.4736	FFD	0.4809
Khiops	0.5078	HellingerBD	0.1965	Ameva	0.5041	Zeta	0.5457	Modified Chi2	0.4697	Zeta	0.4769
HellingerBD	0.4984	Bayesian	0.1941	USD	0.4943	Ameva	0.5456	MODL	0.4620	HellingerBD	0.4729
CACC	0.4961	MODL	0.1918	HDD	0.4878	ID3	0.5403	EqualFrequency	0.4535	USD	0.4560
EqualWidth	0.4909	MDLP	0.1875	ClusterAnalysis	0.4863	HDD	0.5394	DIBD	0.4431	UCPD	0.4552

(continued)

Table 9.6 (continued)

Extended Chi2	0.4766	PKID	0.1846	Zeta	0.4831	MDLP	0.5389	EqualWidth	0.4386	CACC	0.4504
DIBD	0.4759	ID3	0.1818	ID3	0.4769	Distance	0.5368	Extended Chi2	0.4358	MDLP	0.4449
FFD	0.4605	EqualWidth	0.1801	UCPD	0.4763	HellingerBD	0.5353	HDD	0.4048	Distance	0.4429
PKID	0.4526	Modified Chi2	0.1788	MDLP	0.4656	ClusterAnalysis	0.5252	FFD	0.3969	HDD	0.4403
HDD	0.4287	DIBD	0.1778	Distance	0.4470	UCPD	0.5194	PKID	0.3883	ID3	0.4359
USD	0.4282	Chi2	0.1743	CACC	0.4367	CACC	0.5128	USD	0.3845	Extended Chi2	0.4290
ClusterAnalysis	0.4044	IDD	0.1648	IDD	0.4329	Extended Chi2	0.4910	MVD	0.3461	ClusterAnalysis	0.4252
ID3	0.3803	FFD	0.1635	Extended Chi2	0.4226	Bayesian	0.4757	ClusterAnalysis	0.3453	Bayesian	0.3987
IDD	0.3803	ClusterAnalysis	0.1613	Bayesian	0.4201	DIBD	0.4731	Bayesian	0.3419	DIBD	0.3759
MVD	0.3759	Extended Chi2	0.1465	DIBD	0.4167	IDD	0.4618	ID3	0.3241	IDD	0.3650
Bayesian	0.3716	MVD	0.1312	1R	0.3940	1R	0.3980	IDD	0.3066	MVD	0.3446
1R	0.3574	1R	0.1147	MVD	0.3429	MVD	0.3977	1R	0.3004	1R	0.3371
Heter-Disc	0.2709	Heter-Disc	0.1024	Heter-Disc	0.2172	Heter-Disc	0.2583	Heter-Disc	0.2570	Heter-Disc	0.2402
CADD	0.1524	CADD	0.0260	CADD	0.1669	CADD	0.1729	CADD	0.1489	CADD	0.1602

Table 9.7 Wilcoxon test results in number of intervals and inconsistencies

	N. Intervals		Incons. Tra		Incons. Tst	
	+	±	+	±	+	±
1R	10	21	3	17	2	20
Ameva	13	21	6	16	4	21
Bayesian	2	4	10	29	7	29
CACC	7	22	4	17	4	21
CADD	21	28	0	1	0	1
CAIM	14	23	6	19	6	20
Chi2	15	26	9	20	9	20
ChiMerge	15	23	6	20	6	23
ClusterAnalysis	1	4	15	29	9	29
DIBD	21	27	2	7	2	8
Distance	26	28	2	6	2	6
EqualFrequency	7	12	12	26	11	29
EqualWidth	11	18	16	26	13	29
Extended Chi2	14	27	2	14	2	18
FFD	5	8	21	29	16	29
FUSINTER	14	22	11	23	8	29
HDD	0	2	18	29	14	29
HellingerBD	9	15	8	21	7	26
Heter-Disc	29	29	0	1	0	1
ID3	0	1	23	29	16	29
IDD	5	11	8	28	6	29
Khiops	9	15	15	27	12	29
MDLP	22	27	3	9	3	11
Modified Chi2	7	13	17	26	15	29
MODL	5	14	12	24	7	29
MVD	23	28	2	13	2	13
PKID	5	8	22	29	16	29
UCPD	17	25	6	17	5	20
USD	2	4	18	29	15	29
Zeta	12	23	3	9	3	13

discretizers whose performance is within 5 % of the range between the best and the worst method in each measure, that is, $value_{best} - (0.05 \cdot (value_{best} - value_{worst}))$. They should be considered as outstanding methods in each category, regardless of their specific position in the table.

The Wilcoxon test [35, 50, 113] is adopted in this study considering a level of significance equal to $\alpha = 0.05$. Tables 9.7, 9.8 and 9.9 show a summary of all possible

Table 9.8 Wilcoxon test results in accuracy

	C4.5		Data Squeezer		KNN		Naïve Bayes		PUBLIC		Ripper	
	+	±	+	±	+	±	+	±	+	±	+	±
1R	1	12	3	23	2	19	1	9	1	12	1	11
Ameva	14	29	17	29	8	26	9	29	13	29	9	29
Bayesian	1	9	5	26	2	12	2	11	0	11	2	17
CACC	9	28	16	29	2	18	5	28	9	29	4	26
CADD	0	1	1	22	0	1	0	1	0	6	0	0
CAIM	16	29	16	29	11	28	10	29	16	29	11	28
Chi2	13	29	4	26	6	27	9	29	11	29	19	29
ChiMerge	17	29	18	29	13	28	10	29	17	29	9	28
ClusterAnalysis	1	10	0	12	5	24	6	27	1	11	2	20
DIBD	6	21	8	29	2	9	2	8	9	23	1	5
Distance	13	29	16	29	2	17	7	26	13	28	2	13
EqualFrequency	10	27	3	21	18	29	9	29	10	26	11	27
EqualWidth	7	20	2	18	11	28	8	29	6	20	9	27
Extended Chi2	9	27	4	26	3	19	3	17	6	25	2	25
FFD	5	15	0	5	20	28	8	29	1	13	10	27
FUSINTER	21	29	9	29	12	28	15	29	20	29	11	29
HDD	1	18	0	14	4	23	5	28	0	24	7	26
HellingerBD	10	27	4	22	7	26	7	28	10	26	6	26
Heter-Disc	0	9	9	29	0	2	0	3	0	11	1	10
ID3	1	10	0	5	5	22	4	28	0	11	5	26
IDD	1	10	3	23	4	21	2	14	0	12	1	16
Khiops	12	27	3	18	18	29	9	29	9	27	11	29
MDLP	14	29	14	29	3	22	8	29	15	29	2	16
Modified Chi2	11	27	3	21	17	28	10	29	9	29	23	29
MODL	12	28	5	23	14	28	9	29	10	28	17	29
MVD	1	15	5	29	1	8	1	7	0	19	1	13
PKID	5	15	0	6	27	29	9	29	1	13	15	29
UCPD	14	29	7	26	4	17	2	15	14	28	3	19
USD	1	13	3	19	6	23	6	29	1	19	7	25
Zeta	14	29	17	29	4	20	9	29	14	29	7	27

comparisons involved in the Wilcoxon test among all discretizers and measures, for number of intervals and inconsistency rate, accuracy and kappa respectively. Again, the individual comparisons between all possible discretizers are exhibited in the aforementioned URL, where a detailed report of statistical results can be found for each measure and classifier. Tables 9.7, 9.8 and 9.9 summarize, for each method in the rows, the number of discretizers outperformed by using the Wilcoxon test under the

Table 9.9 Wilcoxon test results in kappa

	C4.5		Data Squeezer		KNN		Naïve Bayes		PUBLIC		Ripper	
	+	±	+	±	+	±	+	±	+	±	+	±
1R	1	11	0	15	2	16	2	8	1	13	1	11
Ameva	15	29	24	29	11	26	11	29	16	29	9	29
Bayesian	1	8	1	24	2	10	2	8	1	11	2	17
CACC	11	28	25	29	3	16	7	25	13	29	4	26
CADD	0	1	0	3	0	1	0	1	0	2	0	0
CAIM	17	29	22	29	13	28	11	29	21	29	11	28
Chi2	14	29	2	24	11	27	10	29	13	29	19	29
ChiMerge	19	29	22	29	13	28	11	29	18	29	9	28
ClusterAnalysis	2	10	2	21	5	23	6	22	1	11	2	20
DIBD	8	20	1	24	2	10	2	7	7	18	1	5
Distance	16	29	1	26	2	16	7	28	16	29	2	13
EqualFrequency	11	25	3	25	18	29	10	29	10	23	11	27
EqualWidth	7	20	2	23	14	27	8	28	6	18	9	27
Extended Chi2	10	27	1	20	2	17	3	16	6	23	2	25
FFD	6	14	1	19	23	28	12	29	2	14	10	27
FUSINTER	21	29	16	29	14	28	18	29	19	29	11	29
HDD	2	17	5	25	5	22	6	25	1	22	7	26
HellingerBD	11	23	4	23	9	26	7	21	11	24	6	26
Heter-Disc	0	6	0	12	0	2	0	2	0	8	1	10
ID3	1	8	2	22	4	20	6	26	0	10	5	26
IDD	1	9	1	23	2	18	2	15	1	11	1	16
Khiops	11	24	5	24	18	29	10	29	13	25	11	29
MDLP	16	29	1	24	6	22	8	29	19	29	2	16
Modified Chi2	12	27	1	21	17	27	14	29	9	28	23	29
MODL	12	28	4	24	14	27	12	29	11	28	17	29
MVD	1	12	0	19	1	10	1	6	1	16	1	13
PKID	5	14	2	23	27	29	14	29	2	14	15	29
UCPD	14	29	15	28	4	16	4	16	13	25	3	19
USD	4	13	9	25	6	23	6	25	3	15	7	25
Zeta	15	29	9	27	3	18	6	27	16	29	7	27

column represented by the '+' symbol. The column with the '±' symbol indicates the number of wins and ties obtained by the method in the row. The maximum value for each column is highlighted by a shaded cell.

Once the results are presented in the mentioned tables and graphics, we can stress some interesting properties observed from them, and we can point out the best performing discretizers:

- Regarding the number of intervals, the discretizers which divide the numerical attributes in fewer intervals are *Heter-Disc*, *MVD* and *Distance*, whereas discretizers which require a large number of cut points are *HDD*, *ID3* and *Bayesian*. The Wilcoxon test confirms that *Heter-Disc* is the discretizer that obtains the least intervals outperforming the rest.
- The inconsistency rate both in training data and test data follows a similar trend for all discretizers, considering that the inconsistency obtained in test data is always lower than in training data. *ID3* is the discretizer that obtains the lowest average inconsistency rate in training and test data, albeit the Wilcoxon test cannot find significant differences between it and the other two discretizers: *FFD* and *PKID*. We can observe a close relationship between the number of intervals produced and the inconsistency rate, where discretizers that compute fewer cut points are usually those which have a high inconsistency rate. They risk the consistency of the data in order to simplify the result, although the consistency is not usually correlated with the accuracy, as we will see below.
- In decision trees (*C4.5* and *PUBLIC*), a subset of discretizers can be stressed as the best performing ones. Considering average accuracy, *FUSINTER*, *ChiMerge* and *CAIM* stand out from the rest. Considering average kappa, *Zeta* and *MDLP* are also added to this subset. The Wilcoxon test confirms this result and adds another discretizer, *Distance*, which outperforms 16 of the 29 methods. All methods emphasized are supervised, incremental (except *Zeta*) and use statistical and information measures as evaluators. Splitting/Merging and Local/Global properties have no effect on decision trees.
- Considering rule induction (*DataSqueezer* and *Ripper*), the best performing discretizers are *Distance*, *Modified Chi2*, *Chi2*, *PKID* and *MODL* in average accuracy and *CACC*, *Ameva*, *CAIM* and *FUSINTER* in average kappa. In this case, the results are very irregular due to the fact that the Wilcoxon test emphasizes the *ChiMerge* as the best performing discretizer for *DataSqueezer* instead of *Distance* and incorporates *Zeta* in the subset. With *Ripper*, the Wilcoxon test confirms the results obtained by averaging accuracy and kappa. It is difficult to discern a common set of properties that define the best performing discretizers due to the fact that rule induction methods differ in their operation to a greater extent than decision trees. However, we can say that, in the subset of best methods, incremental and supervised discretizers predominate in the statistical evaluation.
- Lazy and bayesian learning can be analyzed together, due to the fact that the HVDM distance used in KNN is highly related to the computation of bayesian probabilities considering attribute independence [114]. With respect to lazy and bayesian learning, *KNN* and *Naïve Bayes*, the subset of remarkable discretizers is formed by *PKID*, *FFD*, *Modified Chi2*, *FUSINTER*, *ChiMerge*, *CAIM*, *EqualWidth* and *Zeta*, when average accuracy is used; and *Chi2*, *Khiops*, *EqualFrequency* and *MODL* must be added when average kappa is considered. The statistcal report by Wilcoxon informs us of the existence of two outstanding methods: *PKID* for *KNN*, which outperforms 27/29 and *FUSINTER* for *Naïve Bayes*. Here, supervised and unsupervised, direct and incremental, binning and statistical/information evalua-

tion are characteristics present in the best performing methods. However, we can see that all of them are global, thus identifying a trend towards binning methods.

- In general, accuracy and kappa performance registered by discretizers do not differ too much. The behavior in both evaluation metrics are quite similar, taking into account that the differences in kappa are usually lower due to the compensation of random success offered by it. Surprisingly, in *DataSqueezer*, accuracy and kappa offer the greatest differences in behavior, but they are motivated by the fact that this method focuses on obtaining simple rule sets, leaving precision in the background.
- It is obvious that there is a direct dependence between discretization and the classifier used. We have pointed out that a similar behavior in decision trees and lazy/bayesian learning can be detected, whereas in rule induction learning, the operation of the algorithm conditions the effectiveness of the discretizer. Knowing a subset of suitable discretizers for each type of discretizer is a good starting point to understand and propose improvements in the area.
- Another interesting observation can be made about the relationship between accuracy and the number of intervals yielded by a discretizer. A discretizer that computes few cut points does not have to obtain poor results in accuracy and vice versa.
- Finally, we can stress a subset of global best discretizers considering a trade-off between the number of intervals and accuracy obtained. In this subset, we can include *FUSINTER*, *Distance*, *Chi2*, *MDLP* and *UCPD*.

On the other hand, an analysis centered on the 30 discretizers studied is given as follows:

- Many classic discretizers are usually the best performing ones. This is the case of *ChiMerge*, *MDLP*, *Zeta*, *Distance* and *Chi2*.
- Other classic discretizers are not as good as they should be, considering that they have been improved over the years: *EqualWidth*, *EqualFrequency*, *1R*, *ID3* (the static version is much worse than the dynamic inserted in C4.5 operation), *CADD*, *Bayesian* and *ClusterAnalysis*.
- Slight modifications of classic methods have greatly enhanced their results, such as, for example, *FUSINTER*, *Modified Chi2*, *PKID* and *FFD*; but in other cases, the extensions have diminished their performance: *USD*, *Extended Chi2*.
- Promising techniques that have been evaluated under unfavorable circumstances are *MVD* and *UCP*, which are unsupervised methods useful for application to other DM problems apart from classification.
- Recent proposed methods that have been demonstrated to be competitive compared with classic methods and even outperforming them in some scenarios are *Khiops*, *CAIM*, *MODL*, *Ameva* and *CACC*. However, recent proposals that have reported bad results in general are *Heter-Disc*, *HellingerBD*, *DIBD*, *IDD* and *HDD*.
- Finally, this study involves a higher number of data sets than the quantity considered in previous works and the conclusions achieved are impartial towards an specific discretizer. However, we have to stress some coincidences with the conclusions of these previous works. For example in [105], the authors propose

an improved version of *Chi2* in terms of accuracy, removing the user parameter choice. We check and measure the actual improvement. In [122], the authors develop an intense theoretical and analytical study concerning *Naïve Bayes* and propose *PKID* and *FFD* according to their conclusions. Thus, we corroborate that *PKID* is the best suitable method for *Naïve Bayes* and even for *KNN*. Finally, we may note that *CAIM* is one of the simplest discretizers and its effectiveness has also been shown in this study.

References

1. Agrawal, R., Srikant, R.: Fast algorithms for mining association rules. In: Proceedings of the 20th Very Large Data Bases conference (VLDB), pp. 487–499 (1994)
2. Aha, D.W. (ed.): Lazy Learning. Springer, New York (2010)
3. Alcalá-Fdez, J., Fernández, A., Luengo, J., Derrac, J., García, S., Sánchez, L., Herrera, F.: KEEL data-mining software tool: data set repository, integration of algorithms and experimental analysis framework. J. Multiple-Valued Logic Soft Comput. **17**(2–3), 255–287 (2011)
4. Alcalá-Fdez, J., Sánchez, L., García, S., del Jesus, M.J., Ventura, S., Garrell, J.M., Otero, J., Romero, C., Bacardit, J., Rivas, V.M., Fernández, J.C., Herrera, F.: KEEL: a software tool to assess evolutionary algorithms for data mining problems. Soft Comput. **13**(3), 307–318 (2009)
5. An, A., Cercone, N.: Discretization of Continuous Attributes for Learning Classification Rules. In: Proceedings of the Third Conference on Methodologies for Knowledge Discovery and Data Mining, pp. 509–514 (1999)
6. Au, W.H., Chan, K.C.C., Wong, A.K.C.: A fuzzy approach to partitioning continuous attributes for classification. IEEE Trans. Knowl. Data Eng. **18**(5), 715–719 (2006)
7. Augasta, M.G., Kathirvalavakumar, T.: A new discretization algorithm based on range coefficient of dispersion and skewness for neural networks classifier. Appl. Soft Comput. **12**(2), 619–625 (2012)
8. Bache, K., Lichman, M.: UCI machine learning repository (2013). http://archive.ics.uci.edu/ml
9. Bakar, A.A., Othman, Z.A., Shuib, N.L.M.: Building a new taxonomy for data discretization techniques. In: Proceedings on Conference on Data Mining and Optimization (DMO), pp. 132–140 (2009)
10. Bay, S.D.: Multivariate discretization for set mining. Knowl. Inf. Syst. **3**, 491–512 (2001)
11. Berka, P., Bruha, I.: Empirical comparison of various discretization procedures. Int. J. Pattern Recognit. Artif. Intell. **12**(7), 1017–1032 (1998)
12. Berrado, A., Runger, G.C.: Supervised multivariate discretization in mixed data with random forests. In: ACS/IEEE International Conference on Computer Systems and Applications (ICCSA), pp. 211–217 (2009)
13. Berzal, F., Cubero, J.C., Marín, N., Sánchez, D.: Building multi-way decision trees with numerical attributes. Inform. Sci. **165**, 73–90 (2004)
14. Bondu, A., Boulle, M., Lemaire, V.: A non-parametric semi-supervised discretization method. Knowl. Inf. Syst. **24**, 35–57 (2010)
15. Boulle, M.: Khiops: a statistical discretization method of continuous attributes. Mach. Learn. **55**, 53–69 (2004)
16. Boullé, M.: MODL: a bayes optimal discretization method for continuous attributes. Mach. Learn. **65**(1), 131–165 (2006)
17. Breiman, L., Friedman, J., Stone, C.J., Olshen, R.A.: Classification and Regression Trees. Chapman and Hall/CRC, New York (1984)
18. Butterworth, R., Simovici, D.A., Santos, G.S., Ohno-Machado, L.: A greedy algorithm for supervised discretization. J. Biomed. Inform. **37**, 285–292 (2004)

19. Catlett, J.: On changing continuous attributes into ordered discrete attributes. In European Working Session on Learning (EWSL), Lecture Notes on Computer Science, vol. 482, pp. 164–178. Springer (1991)
20. Cerquides, J., Mantaras, R.L.D.: Proposal and empirical comparison of a parallelizable distance-based discretization method. In: Proceedings of the Third International Conference on Knowledge Discovery and Data Mining (KDD), pp. 139–142 (1997)
21. Chan, C., Batur, C., Srinivasan, A.: Determination of quantization intervals in rule based model for dynamic systems. In: Proceedings of the Conference on Systems and Man and and Cybernetics, pp. 1719–1723 (1991)
22. Chao, S., Li, Y.: Multivariate interdependent discretization for continuous attribute. Proc. Third Int. Conf. Inf. Technol. Appl. (ICITA) **2**, 167–172 (2005)
23. Chen, C.W., Li, Z.G., Qiao, S.Y., Wen, S.P.: Study on discretization in rough set based on genetic algorithm. In: Proceedings of the Second International Conference on Machine Learning and Cybernetics (ICMLC), pp. 1430–1434 (2003)
24. Ching, J.Y., Wong, A.K.C., Chan, K.C.C.: Class-dependent discretization for inductive learning from continuous and mixed-mode data. IEEE Trans. Pattern Anal. Mach. Intell. **17**, 641–651 (1995)
25. Chlebus, B., Nguyen, S.H.: On finding optimal discretizations for two attributes. Lect. Notes Artif. Intell. **1424**, 537–544 (1998)
26. Chmielewski, M.R., Grzymala-Busse, J.W.: Global discretization of continuous attributes as preprocessing for machine learning. Int. J. Approximate Reasoning **15**(4), 319–331 (1996)
27. Chou, P.A.: Optimal partitioning for classification and regression trees. IEEE Trans. Pattern Anal. Mach. Intell. **13**, 340–354 (1991)
28. Cios, K.J., Kurgan, L.A., Dick, S.: Highly scalable and robust rule learner: performance evaluation and comparison. IEEE Trans. Syst. Man Cybern. Part B **36**, 32–53 (2006)
29. Cios, K.J., Pedrycz, W., Swiniarski, R.W., Kurgan, L.A.: Data Mining: A Knowledge Discovery Approach. Springer, New York (2007)
30. Clarke, E.J., Barton, B.A.: Entropy and MDL discretization of continuous variables for bayesian belief networks. Int. J. Intell. Syst. **15**, 61–92 (2000)
31. Cohen, J.A.: Coefficient of agreement for nominal scales. Educ. Psychol. Measur. **20**, 37–46 (1960)
32. Cohen, W.W.: Fast Effective Rule Induction. In: Proceedings of the Twelfth International Conference on Machine Learning (ICML), pp. 115–123 (1995)
33. Dai, J.H.: A genetic algorithm for discretization of decision systems. In: Proceedings of the Third International Conference on Machine Learning and Cybernetics (ICMLC), pp. 1319–1323 (2004)
34. Dai, J.H., Li, Y.X.: Study on discretization based on rough set theory. In: Proceedings of the First International Conference on Machine Learning and Cybernetics (ICMLC), pp. 1371–1373 (2002)
35. Demšar, J.: Statistical comparisons of classifiers over multiple data sets. J. Mach. Learn. Res. **7**, 1–30 (2006)
36. Dougherty, J., Kohavi, R., Sahami, M.: Supervised and unsupervised discretization of continuous features. In: Proceedings of the Twelfth International Conference on Machine Learning (ICML), pp. 194–202 (1995)
37. Elomaa, T., Kujala, J., Rousu, J.: Practical approximation of optimal multivariate discretization. In: Proceedings of the 16th International Symposium on Methodologies for Intelligent Systems (ISMIS), pp. 612–621 (2006)
38. Elomaa, T., Rousu, J.: General and efficient multisplitting of numerical attributes. Mach. Learn. **36**, 201–244 (1999)
39. Elomaa, T., Rousu, J.: Necessary and sufficient pre-processing in numerical range discretization. Knowl. Inf. Syst. **5**, 162–182 (2003)
40. Elomaa, T., Rousu, J.: Efficient multisplitting revisited: Optima-preserving elimination of partition candidates. Data Min. Knowl. Disc. **8**, 97–126 (2004)

41. Fayyad, U.M., Irani, K.B.: Multi-interval discretization of continuous-valued attributes for classification learning. In: Proceedings of the 13th International Joint Conference on Artificial Intelligence (IJCAI), pp. 1022–1029 (1993)
42. Ferrandiz, S., Boullé, M.: Multivariate discretization by recursive supervised bipartition of graph. In: Proceedings of the 4th Conference on Machine Learning and Data Mining (MLDM), pp. 253–264 (2005)
43. Flores, J.L., Inza, I., Larrañaga, P.: Larra: Wrapper discretization by means of estimation of distribution algorithms. Intell. Data Anal. **11**(5), 525–545 (2007)
44. Flores, M.J., Gámez, J.A., Martínez, A.M., Puerta, J.M.: Handling numeric attributes when comparing bayesian network classifiers: does the discretization method matter? Appl. Intell. **34**, 372–385 (2011)
45. Friedman, N., Goldszmidt, M.: Discretizing continuous attributes while learning bayesian networks. In: Proceedings of the 13th International Conference on Machine Learning (ICML), pp. 157–165 (1996)
46. Gaddam, S.R., Phoha, V.V., Balagani, K.S.: K-Means+ID3: a novel method for supervised anomaly detection by cascading k-means clustering and ID3 decision tree learning methods. IEEE Trans. Knowl. Data Eng. **19**, 345–354 (2007)
47. Gama, J., Torgo, L., Soares, C.: Dynamic discretization of continuous attributes. In: Proceedings of the 6th Ibero-American Conference on AI: Progress in Artificial Intelligence, IBERAMIA, pp. 160–169 (1998)
48. Garcia, E.K., Feldman, S., Gupta, M.R., Srivastava, S.: Completely lazy learning. IEEE Trans. Knowl. Data Eng. **22**, 1274–1285 (2010)
49. García, M.N.M., Lucas, J.P., Batista, V.F.L., Martín, M.J.P.: Multivariate discretization for associative classification in a sparse data application domain. In: Proceedings of the 5th International Conference on Hybrid Artificial Intelligent Systems (HAIS), pp. 104–111 (2010)
50. García, S., Herrera, F.: An extension on "statistical comparisons of classifiers over multiple data sets" for all pairwise comparisons. J. Mach. Learn. Res. **9**, 2677–2694 (2008)
51. García, S., Luengo, J., Sáez, J.A., López, V., Herrera, F.: A survey of discretization techniques: taxonomy and empirical analysis in supervised learning. IEEE Trans. Knowl. Data Eng. **25**(4), 734–750 (2013)
52. Giráldez, R., Aguilar-Ruiz, J., Riquelme, J., Ferrer-Troyano, F., Rodríguez-Baena, D.: Discretization oriented to decision rules generation. Frontiers Artif. Intell. Appl. **82**, 275–279 (2002)
53. González-Abril, L., Cuberos, F.J., Velasco, F., Ortega, J.A.: AMEVA: an autonomous discretization algorithm. Expert Syst. Appl. **36**, 5327–5332 (2009)
54. Grzymala-Busse, J.W.: A multiple scanning strategy for entropy based discretization. In: Proceedings of the 18th International Symposium on Foundations of Intelligent Systems, ISMIS, pp. 25–34 (2009)
55. Grzymala-Busse, J.W., Stefanowski, J.: Three discretization methods for rule induction. Int. J. Intell. Syst. **16**(1), 29–38 (2001)
56. Gupta, A., Mehrotra, K.G., Mohan, C.: A clustering-based discretization for supervised learning. Stat. Probab. Lett. **80**(9–10), 816–824 (2010)
57. He, H., Garcia, E.A.: Learning from imbalanced data. IEEE Trans. Knowl. Data Eng. **21**(9), 1263–1284 (2009)
58. Ho, K.M., Scott, P.D.: Zeta: A global method for discretization of continuous variables. In: Proceedings of the Third International Conference on Knowledge Discovery and Data Mining (KDD), pp. 191–194 (1997)
59. Holte, R.C.: Very simple classification rules perform well on most commonly used datasets. Mach. Learn. **11**, 63–90 (1993)
60. Hong, S.J.: Use of contextual information for feature ranking and discretization. IEEE Trans. Knowl. Data Eng. **9**, 718–730 (1997)
61. Hu, H.W., Chen, Y.L., Tang, K.: A dynamic discretization approach for constructing decision trees with a continuous label. IEEE Trans. Knowl. Data Eng. **21**(11), 1505–1514 (2009)

62. Ishibuchi, H., Yamamoto, T., Nakashima, T.: Fuzzy data mining: Effect of fuzzy discretization. In: IEEE International Conference on Data Mining (ICDM), pp. 241–248 (2001)
63. Janssens, D., Brijs, T., Vanhoof, K., Wets, G.: Evaluating the performance of cost-based discretization versus entropy- and error-based discretization. Comput. Oper. Res. **33**(11), 3107–3123 (2006)
64. Jiang, F., Zhao, Z., Ge, Y.: A supervised and multivariate discretization algorithm for rough sets. In: Proceedings of the 5th international conference on Rough set and knowledge technology, RSKT, pp. 596–603 (2010)
65. Jiang, S., Yu, W.: A local density approach for unsupervised feature discretization. In: Proceedings of the 5th International Conference on Advanced Data Mining and Applications, ADMA, pp. 512–519 (2009)
66. Jin, R., Breitbart, Y., Muoh, C.: Data discretization unification. Knowl. Inf. Syst. **19**, 1–29 (2009)
67. Kang, Y., Wang, S., Liu, X., Lai, H., Wang, H., Miao, B.: An ICA-based multivariate discretization algorithm. In: Proceedings of the First International Conference on Knowledge Science, Engineering and Management (KSEM), pp. 556–562 (2006)
68. Kerber, R.: Chimerge: Discretization of numeric attributes. In: National Conference on Artifical Intelligence American Association for Artificial Intelligence (AAAI), pp. 123–128 (1992)
69. Kononenko, I., Sikonja, M.R.: Discretization of continuous attributes using relieff. In: Proceedings of Elektrotehnika in Racunalnika Konferenca (ERK) (1995)
70. Kurgan, L.A., Cios, K.J.: CAIM discretization algorithm. IEEE Trans. Knowl. Data Eng. **16**(2), 145–153 (2004)
71. Kurtcephe, M., Güvenir, H.A.: A discretization method based on maximizing the area under receiver operating characteristic curve. Int. J. Pattern Recognit. Artif. Intell. **27**(1), 8 (2013)
72. Lee, C.H.: A hellinger-based discretization method for numeric attributes in classification learning. Knowl. Based Syst. **20**, 419–425 (2007)
73. Li, R.P., Wang, Z.O.: An entropy-based discretization method for classification rules with inconsistency checking. In: Proceedings of the First International Conference on Machine Learning and Cybernetics (ICMLC), pp. 243–246 (2002)
74. Li, W.L., Yu, R.H., Wang, X.Z.: Discretization of continuous-valued attributes in decision tree generation. In: Proocedings of the Second International Conference on Machine Learning and Cybernetics (ICMLC), pp. 194–198 (2010)
75. Liu, H., Hussain, F., Tan, C.L., Dash, M.: Discretization: an enabling technique. Data Min. Knowl. Disc. **6**(4), 393–423 (2002)
76. Liu, H., Setiono, R.: Feature selection via discretization. IEEE Trans. Knowl. Data Eng. **9**, 642–645 (1997)
77. Liu, L., Wong, A.K.C., Wang, Y.: A global optimal algorithm for class-dependent discretization of continuous data. Intell. Data Anal. **8**, 151–170 (2004)
78. Liu, X., Wang, H.: A discretization algorithm based on a heterogeneity criterion. IEEE Trans. Knowl. Data Eng. **17**, 1166–1173 (2005)
79. Ludl, M.C., Widmer, G.: Relative unsupervised discretization for association rule mining, In: Proceedings of the 4th European Conference on Principles of Data Mining and Knowledge Discovery, The Fourth European Conference on Principles and Practice of Knowledge Discovery in Databases (PKDD), pp. 148–158 (2000)
80. Macskassy, S.A., Hirsh, H., Banerjee, A., Dayanik, A.A.: Using text classifiers for numerical classification. In: Proceedings of the 17th International Joint Conference on Artificial Intelligence, Vol. 2 (IJCAI), pp. 885–890 (2001)
81. Mehta, S., Parthasarathy, S., Yang, H.: Toward unsupervised correlation preserving discretization. IEEE Trans. Knowl. Data Eng. **17**, 1174–1185 (2005)
82. Monti, S., Cooper, G.: A latent variable model for multivariate discretization. In: Proceedings of the Seventh International Workshop on AI & Statistics (Uncertainty) (1999)
83. Monti, S., Cooper, G.F.: A multivariate discretization method for learning bayesian networks from mixed data. In: Proceedings on Uncertainty in Artificial Intelligence (UAI), pp. 404–413 (1998)

84. Muhlenbach, F., Rakotomalala, R.: Multivariate supervised discretization, a neighborhood graph approach. In: Proceedings of the 2002 IEEE International Conference on Data Mining, ICDM, pp. 314–320 (2002)
85. Nemmiche-Alachaher, L.: Contextual approach to data discretization. In: Proceedings of the International Multi-Conference on Computing in the Global Information Technology (ICCGI), pp. 35–40 (2010)
86. Nguyen, S.H., Skowron, A.: Quantization of real value attributes - rough set and boolean reasoning approach. In: Proceedings of the Second Joint Annual Conference on Information Sciences (JCIS), pp. 34–37 (1995)
87. Pazzani, M.J.: An iterative improvement approach for the discretization of numeric attributes in bayesian classifiers. In: Proceedings of the First International Conference on Knowledge Discovery and Data Mining (KDD), pp. 228–233 (1995)
88. Perner, P., Trautzsch, S.: Multi-interval discretization methods for decision tree learning. In: Advances in Pattern Recognition, Joint IAPR International Workshops SSPR 98 and SPR 98, pp. 475–482 (1998)
89. Pfahringer, B.: Compression-based discretization of continuous attributes. In: Proceedings of the 12th International Conference on Machine Learning (ICML), pp. 456–463 (1995)
90. Pongaksorn, P., Rakthanmanon, T., Waiyamai, K.: DCR: Discretization using class information to reduce number of intervals. In: Proceedings of the International Conference on Quality issues, measures of interestingness and evaluation of data mining model (QIMIE), pp. 17–28 (2009)
91. Qu, W., Yan, D., Sang, Y., Liang, H., Kitsuregawa, M., Li, K.: A novel chi2 algorithm for discretization of continuous attributes. In: Proceedings of the 10th Asia-Pacific web conference on Progress in WWW research and development, APWeb, pp. 560–571 (2008)
92. Quinlan, J.R.: C4.5: Programs for Machine Learning. Morgan Kaufmann Publishers Inc, San Francisco (1993)
93. Rastogi, R., Shim, K.: PUBLIC: a decision tree classifier that integrates building and pruning. Data Min. Knowl. Disc. **4**, 315–344 (2000)
94. Richeldi, M., Rossotto, M.: Class-driven statistical discretization of continuous attributes. In: Proceedings of the 8th European Conference on Machine Learning (ECML), ECML '95, pp. 335–338 (1995)
95. Roy, A., Pal, S.K.: Fuzzy discretization of feature space for a rough set classifier. Pattern Recognit. Lett. **24**, 895–902 (2003)
96. Ruiz, F.J., Angulo, C., Agell, N.: IDD: a supervised interval distance-based method for discretization. IEEE Trans. Knowl. Data Eng. **20**(9), 1230–1238 (2008)
97. Sang, Y., Jin, Y., Li, K., Qi, H.: UniDis: a universal discretization technique. J. Intell. Inf. Syst. **40**(2), 327–348 (2013)
98. Sang, Y., Li, K., Shen, Y.: EBDA: An effective bottom-up discretization algorithm for continuous attributes. In: Proceedings of the 10th IEEE International Conference on Computer and Information Technology (CIT), pp. 2455–2462 (2010)
99. Shehzad, K.: Edisc: a class-tailored discretization technique for rule-based classification. IEEE Trans. Knowl. Data Eng. **24**(8), 1435–1447 (2012)
100. Singh, G.K., Minz, S.: Discretization using clustering and rough set theory. In: Proceedings of the 17th International Conference on Computer Theory and Applications (ICCTA), pp. 330–336 (2007)
101. Su, C.T., Hsu, J.H.: An extended chi2 algorithm for discretization of real value attributes. IEEE Trans. Knowl. Data Eng. **17**, 437–441 (2005)
102. Subramonian, R., Venkata, R., Chen, J.: A visual interactive framework for attribute discretization. In: Proceedings of the First International Conference on Knowledge Discovery and Data Mining (KDD), pp. 82–88 (1997)
103. Sun, Y., Wong, A.K.C., Kamel, M.S.: Classification of imbalanced data: a review. Int. J. Pattern Recognit. Artif. Intell. **23**(4), 687–719 (2009)
104. Susmaga, R.: Analyzing discretizations of continuous attributes given a monotonic discrimination function. Intell. Data Anal. **1**(1–4), 157–179 (1997)

105. Tay, F.E.H., Shen, L.: A modified chi2 algorithm for discretization. IEEE Trans. Knowl. Data Eng. **14**, 666–670 (2002)
106. Tsai, C.J., Lee, C.I., Yang, W.P.: A discretization algorithm based on class-attribute contingency coefficient. Inf. Sci. **178**, 714–731 (2008)
107. Vannucci, M., Colla, V.: Meaningful discretization of continuous features for association rules mining by means of a SOM. In: Proocedings of the 12th European Symposium on Artificial Neural Networks (ESANN), pp. 489–494 (2004)
108. Ventura, D., Martinez, T.R.: BRACE: A paradigm for the discretization of continuously valued data, In: Proceedings of the Seventh Annual Florida AI Research Symposium (FLAIRS), pp. 117–121 (1994)
109. Ventura, D., Martinez, T.R.: An empirical comparison of discretization methods. In: Proceedings of the 10th International Symposium on Computer and Information Sciences (ISCIS), pp. 443–450 (1995)
110. Wang, K., Liu, B.: Concurrent discretization of multiple attributes. In: Proceedings of the Pacific Rim International Conference on Artificial Intelligence (PRICAI), pp. 250–259 (1998)
111. Wang, S., Min, F., Wang, Z., Cao, T.: OFFD: Optimal flexible frequency discretization for naive bayes classification. In: Proceedings of the 5th International Conference on Advanced Data Mining and Applications, ADMA, pp. 704–712 (2009)
112. Wei, H.: A novel multivariate discretization method for mining association rules. In: 2009 Asia-Pacific Conference on Information Processing (APCIP), pp. 378–381 (2009)
113. Wilcoxon, F.: Individual comparisons by ranking methods. Biometrics **1**, 80–83 (1945)
114. Wilson, D.R., Martinez, T.R.: Reduction techniques for instance-based learning algorithms. Mach. Learn. **38**(3), 257–286 (2000)
115. Wong, A.K.C., Chiu, D.K.Y.: Synthesizing statistical knowledge from incomplete mixed-mode data. IEEE Trans. Pattern Anal. Mach. Intell. **9**, 796–805 (1987)
116. Wu, M., Huang, X.C., Luo, X., Yan, P.L.: Discretization algorithm based on difference-similitude set theory. In: Proceedings of the Fourth International Conference on Machine Learning and Cybernetics (ICMLC), pp. 1752–1755 (2005)
117. Wu, Q., Bell, D.A., Prasad, G., McGinnity, T.M.: A distribution-index-based discretizer for decision-making with symbolic ai approaches. IEEE Trans. Knowl. Data Eng. **19**, 17–28 (2007)
118. Wu, Q., Cai, J., Prasad, G., McGinnity, T.M., Bell, D.A., Guan, J.: A novel discretizer for knowledge discovery approaches based on rough sets. In: Proceedings of the First International Conference on Rough Sets and Knowledge Technology (RSKT), pp. 241–246 (2006)
119. Wu, X.: A bayesian discretizer for real-valued attributes. Comput. J. **39**, 688–691 (1996)
120. Wu, X., Kumar, V. (eds.): The Top Ten Algorithms in Data Mining. Data Mining and Knowledge Discovery. Chapman and Hall/CRC, Taylor and Francis, Boca Raton (2009)
121. Yang, P., Li, J.S., Huang, Y.X.: HDD: a hypercube division-based algorithm for discretisation. Int. J. Syst. Sci. **42**(4), 557–566 (2011)
122. Yang, Y., Webb, G.I.: Discretization for naive-bayes learning: managing discretization bias and variance. Mach. Learn. **74**(1), 39–74 (2009)
123. Yang, Y., Webb, G.I., Wu, X.: Discretization methods. In: Data Mining and Knowledge Discovery Handbook, pp. 101–116 (2010)
124. Zhang, G., Hu, L., Jin, W.: Discretization of continuous attributes in rough set theory and its application. In: Proceedings of the 2004 IEEE Conference on Cybernetics and Intelligent Systems (CIS), pp. 1020–1026 (2004)
125. Zhu, W., Wang, J., Zhang, Y., Jia, L.: A discretization algorithm based on information distance criterion and ant colony optimization algorithm for knowledge extracting on industrial database. In: Proceedings of the 2010 IEEE International Conference on Mechatronics and Automation (ICMA), pp. 1477–1482 (2010)
126. Zighed, D.A., Rabaséda, S., Rakotomalala, R.: FUSINTER: a method for discretization of continuous attributes. Int. J. Uncertainty, Fuzziness Knowl. Based Syst. **6**, 307–326 (1998)
127. Zighed, D.A., Rakotomalala, R., Feschet, F.: Optimal multiple intervals discretization of continuous attributes for supervised learning. In: Proceedings of the First International Conference on Knowledge Discovery and Data Mining (KDD), pp. 295–298 (1997)